# Design of a 6-bit SAR ADC with Aliasing and Anti-Aliasing

EE25BTECH11063 (Yalamati Vejith)
EE25BTECH11064 (Yojit Manral))

May 4, 2026

**Abstract**

This project implements a 6-bit Successive Approximation Register (SAR) ADC using an Arduino Uno, LM339 comparator, 2N7000 MOSFET sample-and-hold, BC547 BJT, and an R-2R resistor ladder DAC.TThe project also successfully demonstrates the effect of aliasing due to undersampling and the role of an anti-aliasing filter in recovering the correct signal.

# 1 Theory

## 1.1 SAR ADC Operating Principle

A SAR ADC converts an analog voltage to a digital code by performing a binary search. Starting from the most significant bit (MSB), it sets each bit to 1 one at a time and compares the resulting DAC output voltage ($V_{DAC}$) against the held input voltage ($V_{in}$). If $V_{DAC} > V_{in}$, the bit is cleared; otherwise it is kept. This process repeats for all $N$ bits, completing one conversion in exactly $N$ clock cycles.

For a 6-bit converter with $V_{ref} = 5$ V, the resolution is:

$$\text{LSB} = \text{step size} = \frac{V_{ref}}{2^N} = \frac{5}{64} \approx 78.1\ \text{mV} \tag{1}$$

The digital output code $D$ for an input voltage $V_{in}$ is:

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} \times (2^N - 1) \right\rfloor = \left\lfloor \frac{V_{in}}{5} \times 63 \right\rfloor, \quad D \in \{0, 1, 2, \ldots, 63\} \tag{2}$$

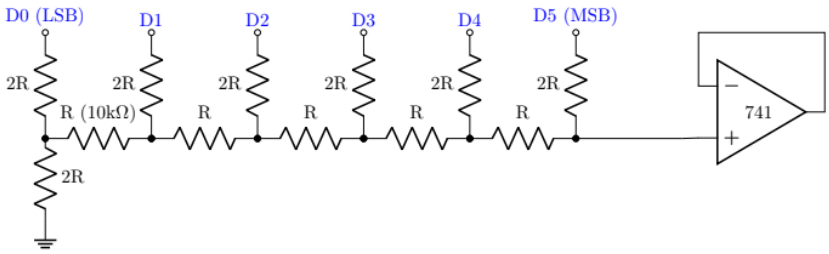
## 1.2 R-2R Ladder DAC

The R-2R ladder converts the Arduino's 6-bit digital output into an analog voltage. Using only two resistor values ($\frac{47}{2}$ kΩ and 47 kΩ), the output voltage is:

$$V_{DAC} = V_{ref} \times \frac{D}{2^N} = 5 \times \frac{D}{64} \tag{3}$$

where

$$D = 2^5 \times D_5 + 2^4 \times D_4 + 2^3 \times D_3 + 2^2 \times D_2 + 2^1 \times D_1 + 2^0 \times D_0 \tag{4}$$




DAC circuit diagram

## 1.3 Sample and Hold

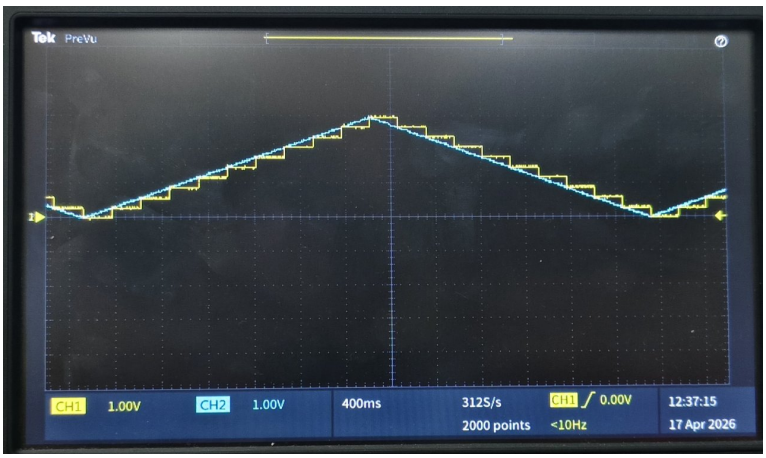
The sample-and-hold captures the instantaneous input voltage during the sampling phase and holds it constant during the conversion. The RC time constant of the 2N7000 switch (Ron ≈ 100 Ω) and the 220 nF hold capacitor gives:

$$\tau = R_{on} \times C = 100\,\Omega \times 220\,\text{nF} = 22\,\mu\text{s} \tag{5}$$

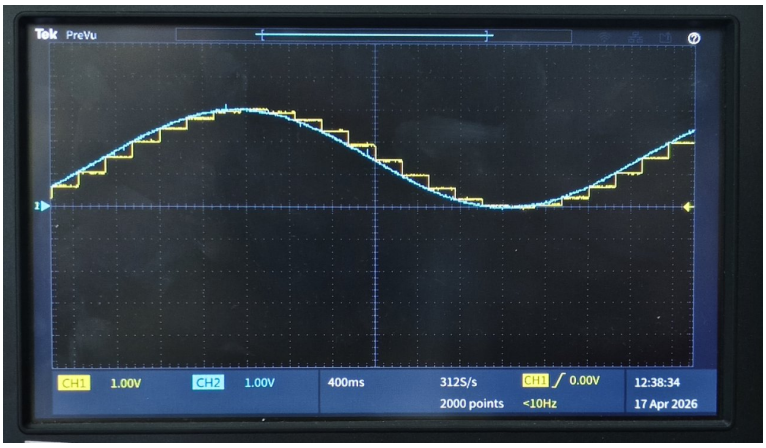
The 2N7000 MOSFET was used as the sampling switch:

- Drain: connected to the analog input signal.
- Source: connected to one leg of the 220 nF hold capacitor (other leg to GND).
- Gate: driven by the level shifter output (12 V pulse).
- The source node was also connected to pin 3 of the 741 op-amp (non-inverting input).
- Pin 2 and pin 6 of the 741 were shorted for unity-gain follower configuration.
- 741 powered from ±15 V bench supply.

**Note :**
Due to gate voltage of the N7000 transistor even if the input voltage is 0-5v it samples till input voltage of 3.5v only to avoid this problem we use BC547 as voltage shifter.




Sampling of 0-3v ramp input




Sampling of a sine wave of 0-3v

## 1.4 BC547 Level Shifter

The Arduino outputs only 5 V(as the input is 0-5v increasing ramp) but the 2N7000 requires at least 12 V on the gate to remain fully ON across the entire 0–5 V signal range. A BC547 NPN transistor was wired as a common-emitter inverting level shifter:

- Base: connected to Arduino clock pin through a 3.3 kΩ resistor.
- Emitter: connected to GND.

- Collector: connected to the 2N7000 gate and to +12 V through a 3.3 kΩ pull-up.
- Above used resistance can be anything between 1kΩ and 10kΩ preferably 3.3 kΩ.

This inverts the logic: Arduino HIGH ⇒ BC547 ON ⇒ gate pulled to 0 V ⇒ MOSFET OFF (hold). Arduino LOW ⇒ BC547 OFF ⇒ gate pulled to 12 V ⇒ MOSFET ON (sample).


Sampling of 0-5v ramp input after using BC547

## 1.5 LM339 Comparator

The LM339 was wired as follows:

- Non-inverting input (+): connected to the sample-and-hold output ($V_{in}$ held).
- Inverting input (−): connected to the R-2R DAC output ($V_{DAC}$).
- Output: connected to Arduino pin A0 .
- $V_{CC}$: connected to +15 V bench supply.

The comparator output is open-collector. When $V_{in} > V_{DAC}$ the output is HIGH (Arduino reads 1, bit is kept). When $V_{in} < V_{DAC}$ the output is pulled LOW (Arduino reads 0, bit is cleared).

## 1.6 Arduino Code

```
// 6-BIT SAR ADC CONTROLLER (0-5V RAMP)
// ========================================

// 1. PIN DEFINITIONS
const int clockPin = A1;
const int compPin = A0; // Input taking the IL339 Comparator Output

// Arrays for easy looping (From LSB to MSB)
const int dacPins[] = {2, 3, 4, 5, 6, 7};
const int ledPins[] = {8, 9, 10, 11, 12, 13};

// 2. TIMING VARIABLES
unsigned long previousMillis = 0;
bool isClockHigh = false;
void setup() {
  Serial.begin(9600);

  // Initialize Clock Pin
  pinMode(clockPin, OUTPUT);
  digitalWrite(clockPin, LOW);

  // Initialize Comparator Pin
  pinMode(compPin, INPUT_PULLUP);

  // Initialize DAC and LED pins as outputs
  for (int i = 0; i < 6; i++) {
    pinMode(dacPins[i], OUTPUT);
    pinMode(ledPins[i], OUTPUT);
  }
}

void loop() {
  unsigned long currentMillis = millis();

  // GENERATE THE TRACK & HOLD CLOCK

  if (isClockHigh) {
    // HOLD PHASE: Wait 5ms, then switch to TRACK
    if (currentMillis - previousMillis >= 5) {
      digitalWrite(clockPin, LOW); // Pull LOW to start Tracking
      isClockHigh = false;
      previousMillis = currentMillis; }
  } else {
    // TRACK PHASE: Wait 245ms, then switch to HOLD
    if (currentMillis - previousMillis >= 245) {
      digitalWrite(clockPin, HIGH); // Pull HIGH to start Holding
      isClockHigh = true;
      previousMillis = currentMillis;
      performSARConversion(); }}}
```

```
// ================================================
//  SAR GUESSING, DAC, AND LED OUTPUT
// ================================================
void performSARConversion() {
  int sarResult = 0; // Start with a blank state (000000)


  // Loop from MSB (Bit 5) down to LSB (Bit 0)
  for (int bit = 5; bit >= 0; bit--) {
    // Make a guess (Set the current bit to 1)
    bitSet(sarResult, bit);
    //  Send this 6-bit guess to the physical DAC pins (Generating 0-5V)
    for (int i = 0; i < 6; i++) {
      int bitState = bitRead(sarResult, i);
      digitalWrite(dacPins[i], bitState);
    }

    delayMicroseconds(500);

    //  Take the Comparator Output from Pin A0
    // If LOW: DAC Guess (-) is greater than 5V Ramp (+). The guess was too high.
    // If HIGH: DAC Guess (-) is less than 5V Ramp (+). The guess was good.
    int comparatorState = digitalRead(compPin);

    if (comparatorState == LOW) {
      bitClear(sarResult, bit); // Guess was too high, revert the bit to 0
    }
  }

  // E. Send the final, locked-in binary result to the 6 LEDs
  for (int i = 0; i < 6; i++) {
    int bitState = bitRead(sarResult, i);
    digitalWrite(ledPins[i], bitState);
  }

  // F. Print the final digital value (0 to 63) to the Serial Monitor
  Serial.print("Final Digital Output: ");
  Serial.println(sarResult);
}
```

- **Clock generation:** A 4 Hz asymmetric clock was generated using `millis()` timing. The LOW (sampling) phase lasts 5 ms and the HIGH (hold) phase lasts 245 ms, giving a period of 250 ms. Due to the BC547 inversion, the duty cycle was set to 99% HIGH so the effective sampling pulse is only 5 ms wide.

- **SAR conversion:** Immediately upon entering the hold phase, `performSARConversion()` is called. It performs a 6-cycle binary search, writing each guess to the DAC pins and reading the LM339 output to decide whether to keep or clear each bit. The final 6-bit result is output to the LED pins and printed to the Serial Monitor.

- Input frequency can be anything depending upon the clock frequency it should satisfy Nyquist rule i.e $f_{clk} > 2f_{max}$ where $f_{max}$ is the maximum input frequency

## 1.7 Components

| Component | Specification | Function |
|---|---|---|
| Arduino Uno | ATmega328P | SAR logic and clock generation |
| Comparator | LM339 | Compare $V_{in}$ vs $V_{DAC}$ |
| MOSFET | 2N7000 N-channel | Sample-and-hold switch |
| NPN Transistor | BC547 | 5 V to 12 V level shifter |
| Op-Amp | 741 | Unity-gain buffer for S&H |
| Resistors (R) | 23.5 kΩ | R-2R ladder R rungs |
| Resistors (2R) | 47 kΩ | R-2R ladder 2R rungs |
| Hold capacitor | 220 nF | Voltage storage in S&H |
| Filter capacitor | 1 $\mu$F | Anti-aliasing RC filter |
| LEDs | ×6 | Binary output display (0–63) |
| LED resistors | 330 Ω | Current limiting |
| Pull-up resistor | 3.3 kΩ | LM339 open-collector output |
| Power supply | +15 V, +12 V | LM339 and gate drive rails |

Components used in the 6-bit SAR ADC implementation.

## 1.8 Output Demonstration

- Input wave is an increasing 0-5v ramp wave of frequency 40milliHz and clock frequency 4Hz.

- The expected output is a counter which counts from 0 to 63 and goes back to 0 again.

- Experimental output is recorded and uploaded in the link below

  SAR ADC output demonstration link

## 1.9 Difficulties Faced and Resolutions

### 1.9.1 Output Capped at 3.5 V

- This is due to the gate voltage of N7000 transistor.
- This is resolved by using an BJT BC547 and connecting it according to the abov e pin connections.

### 1.9.2 Inverted Logic After Adding BC547

- After setting BC547 Sampling was reversed at very low duty cycle which caused mosfet on and preventing stable hold.
- This was resolved by inverting the clock logic i.e keeping duty cycle high i.e 99.5%

### 1.9.3 Incorrect SAR Output

- Initially we observed only count of 0-50 in binary instead of 0-63.
- This is due to powering LM339 comparator to 5v instead of 15v.
- BY powering it to 15v we got 0-63 output in binary and it worked.

# 2 Aliasing and Anti-Aliasing

## 2.1 Objective

To demonstrate the phenomenon of aliasing and anti-aliasing in a sampled system by varying the input signal frequency above and below the Nyquist limit, and to observe how an RC anti-aliasing low-pass filter suppresses out-of-band frequency components before sampling. The Fast Fourier Transform (FFT) function of the DSO is used to analyse the frequency content of the sampled signal in each case.

## 2.2 Theory

### 2.2.1 The Nyquist Sampling Theorem

The Nyquist sampling theorem states that a band-limited signal can be perfectly reconstructed from its samples if and only if the sampling frequency $f_s$ is at least twice the highest frequency component $f_{in}$ present in the signal:

$$f_s \geq 2 \cdot f_{in} \tag{6}$$

The quantity $f_s/2$ is called the Nyquist frequency. Any input frequency above this limit cannot be correctly represented by the sampled data.

### 2.2.2 Aliasing

When a signal at frequency $f_{in}$ is sampled at $f_s$ and $f_{in} > f_s/2$, the sampled output appears at a false (aliased) frequency given by:

$$f_{alias} = |f_{in} - n \cdot f_s|, \quad n \in \mathbb{Z} \tag{7}$$

where $n$ is chosen such that $f_{alias} \leq f_s/2$. In the frequency domain (FFT), aliased components appear as spurious peaks at incorrect frequencies, making the digitized signal appear to be a lower-frequency signal than it actually is.

### 2.2.3 Anti-Aliasing Filter

An anti-aliasing filter is a low-pass filter placed before the sampler to attenuate all frequency components above $f_s/2$ before they can be sampled. In this experiment, a first-order RC filter with cutoff frequency:

$$f_c = \frac{1}{2\pi RC} \tag{8}$$

is used. With $R = 820\Omega$ and $C = 1\,\mu$F:

$$f_c = \frac{1}{2\pi \times 820 \times 1 \times 10^{-6}} \approx 194\,\text{Hz} \tag{9}$$

## 2.3 Equipment and Setup

| **Equipment / Component** | **Role** |
|---|---|
| Arbitrary Function Generator (AFG) | Generates sinusoidal input at variable frequency |
| Digital Storage Oscilloscope (DSO) | Captures CH1 (time domain) and CH2 FFT |
| RC Low-Pass Filter | Anti-aliasing filter, $f_c \approx 194\,\text{Hz}$ |
| 2N7000 MOSFET + 220 nF capacitor | Sample-and-hold circuit |
| 741 Op-Amp (unity gain) | Buffer for hold capacitor |
| BC547 + 12 V supply | Level shifter for MOSFET gate drive |

Equipment and components used in the aliasing demonstration.

The DSO was configured as follows:

- **CH1**: Yellow colored waveform of CH1 is connected directly to the input itself.

- **CH2** : Blue colored CH2 is connected to the reconstructed signal

## 2.4 Procedure

1. The sample-and-hold circuit was set up with the 2N7000 MOSFET, 220 nF hold capacitor, and 741 buffer as described in the SAR ADC section.
2. The AFG was connected directly to the sample-and-hold input (no anti-aliasing filter).
3. The input frequency was set to three test values: 100 Hz, 300 Hz, and 500 Hz, and the DSO FFT was captured for each.
4. Clock frequency corresponding to the above input frequencies are 1000 HZ, 600 HZ, 400 Hz.
5. Above 3 cases corresponds to anti- aliasing, borderd aliasing, aliasing.
6. The RC anti-aliasing filter ($f_c \approx 194$ Hz) was then inserted between the input and the sample-and-hold input.
7. All captures were taken with the DSO in FFT mode on CH2 (Hanning window, 20 dB vertical scale, dBV RMS).

## 2.5 Observations and Analysis

### 2.5.1 Case 1: $f_{in}$ = 100 Hz, $f_{clk}$ = 1000 Hz

- Clearly $f_{clk} > f_{in}$ so Nyquist rule is satisfied .
- This corresponds to anti-aliasing cases so there is no need of anti-aliasing filter in this case.
- Signal will be reconstructed perfectly.
- In FFT we can observe 2 peaks the very initial peak is due to noise and the second peak will be observed at input frequency i.e 100Hz
- For the FFT photo captured below 1 Horizontal scale is 156Hz/div.


FFT photo

- Clearly we can see the blue signal which was reconstructed perfectly.
- **Calculation of frequency of 2nd peak :**
  2nd peak appeared just after crossing 3rd divison out of 5 so it is

$$\approx \frac{156 \times 3.2}{5} Hz \tag{10}$$

$$\approx 100 Hz \tag{11}$$

### 2.5.2 Case 2: $f_{in}$ = 300 Hz, $f_{clk}$ = 600 Hz

- Here $f_{clk} = 2f_{in}$ so Nyquist rule is satisfied .
- This corresponds to the border case of aliasing and anti-aliasing
- The Signal will be reconstructed with slight errors.
- In FFT we can observe 2 peaks the very initial peak is due to noise and the second peak will be observed at input frequency i.e ≈300Hz

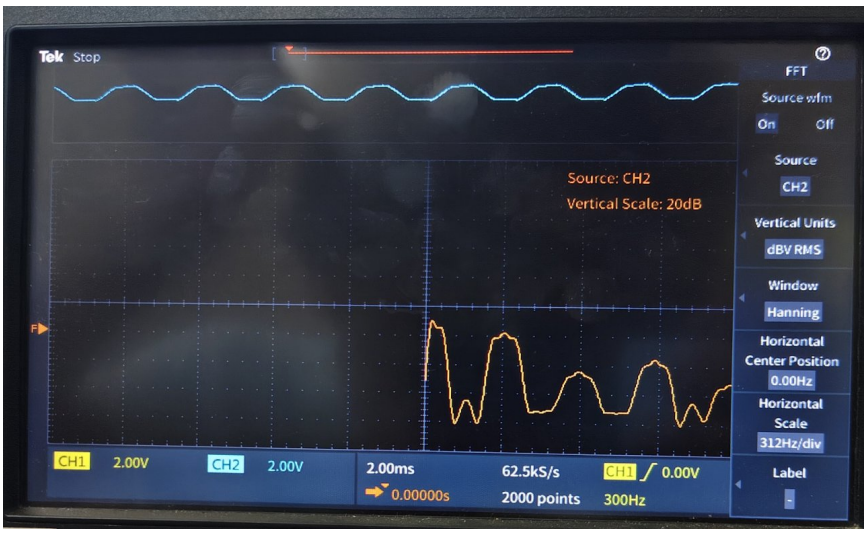
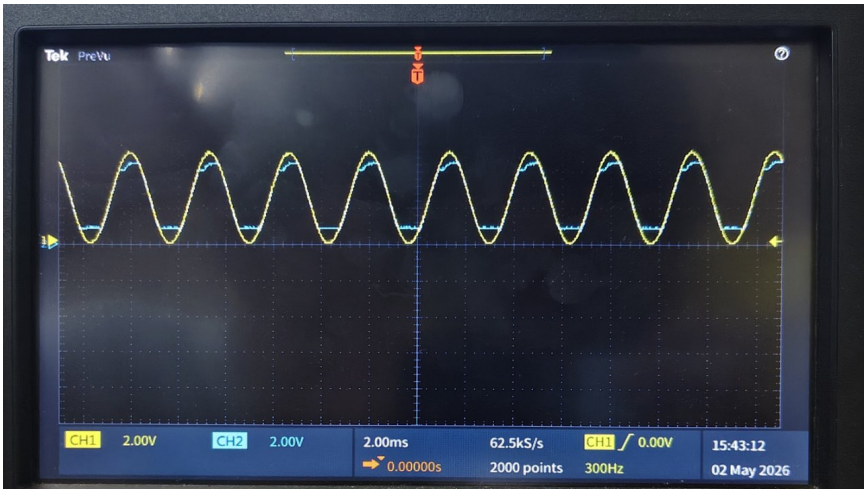



Photo of FFT

- Here we can see that one horizontal scale is 312Hz/div.
- **Calculation of frequency of 2nd peak :**
  2nd peak appeared almost at the 1st horizontal divison so it is

$$\approx 300Hz \tag{12}$$




Original and Reconstructed signal

- Yellow corresponds to original signal and blue corresponds to reconstructed signal

### 2.5.3 Case 3: $f_{in}$ = 500 Hz, $f_{clk}$ = 400 Hz without anti-aliasing filter

- Here $f_{clk} < 2f_{in}$ so Nyquist rule is not satisfied .
- This corresponds to overlapping or aliasing.
- The Signal will not be reconstructed back.
- In FFT we observe many peaks at frequencies like $f_{alias} = |f_{in} - n \cdot f_s|\,, \quad n \in \mathbb{Z}$
- In this particular case we observe many peaks at frequencies like 100Hz, 300Hz,500, etc.


Corresponding FFT

- In this FFT photo captured above 1 Horizontal scale is 156Hz/div.
- Here we can clearly see many peaks at different aliasing frequencies.


Input and reconstructed signal

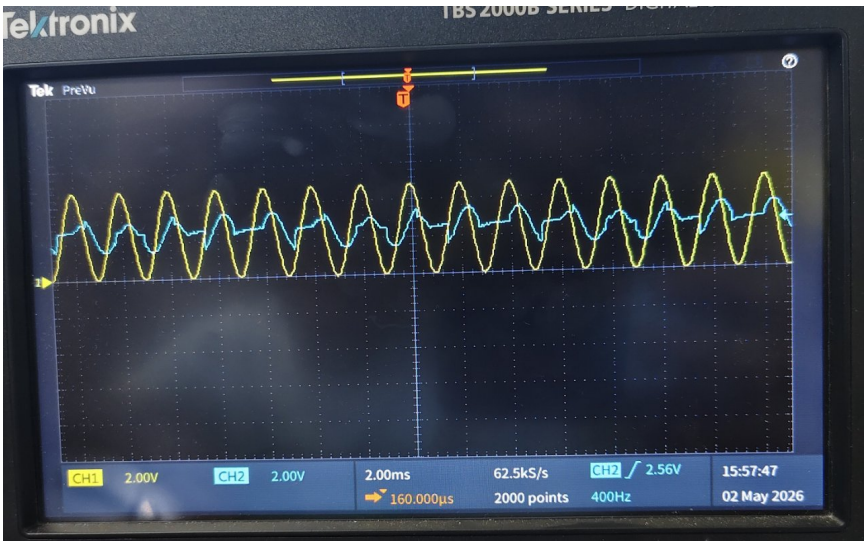
- Yellow corresponds to the input signal and the blue corresponds to the improper reconstructed signal.

### 2.5.4 Case 4: $f_{in}$ = 500 Hz, $f_{clk}$ = 400 Hz with anti-aliasing filter

- Here $f_{clk} < 2f_{in}$ so Nyquist rule is not satisfied .
- This corresponds to overlapping or aliasing.
- The cut off frequency of this filter must be $< \frac{f_s}{2}$
- In this case cutoff frequency is 194Hz which is < 200Hz.
- In this case in FFT we dont observe any significant peaks other than the peak at the start due to noise.
- Here we can reconstruct the signal back but the amplitude of the signal will be reduced due to passing it through anti-aliasing filter and the reconstruction is alos not perfect.


Corresponding FFT

- Here we can clearly observe there are no significant peaks in the FFT other than the peak at the start due to noise.




Input and reconstructed signal

- Yellow corresponds to the input signal and the blue corresponds to the improper reconstructed signal.

**Conclusion**

We were able to build a 6-bit SAR adc and we demonstrated it successfully. We were also to show the effect of aliasing and anti-aliasing also.